

The evaluation uses BoxPush and AI2-THOR-style domains with controlled coverage gaps, irrelevant distractor variables, hidden causal factors, object-class ambiguities, and task-dependent constraints. Metrics include planning efficiency, coverage-gap detection, abstraction soundness, false expansion rate, recovery usefulness, model size, and downstream task success.

The expected contribution is an adaptive model-management layer that makes symbolic coverage both compact and expandable.

4.8 TBD / Software Engineering Final Project — Trace Review and Artifact QA Workbench

This project builds the observability and quality-assurance infrastructure needed by the research components. The workbench ingests logged executions and presents them as inspectable timelines containing user tasks, symbolic goals, selected skills, observations, belief snapshots, monitor states, translator residuals, divergence reports, guard decisions, recovery actions, and final explanations.

The tool provides side-by-side views of the NL/VLM and symbolic states, a divergence-report browser, an evidence viewer that links explanations to logged events, an artifact registry for skill contracts, monitors, alias tables, translators, trust levels, provenance, and test status, and a dashboard for perturbation-test results.

The workbench supports mock traces and curated replay scenarios, including clean execution, closed-receptacle recovery, observation contradiction, coverage gap, stale observation, alias mismatch, and hard-constraint violation.

This project is intentionally infrastructure-oriented. It does not implement the planner, DBN learner, monitor synthesis method, DSPy optimizer, VLM decoder, or orchestration policy. Instead, it provides the debugging, replay, regression testing, artifact review, and demonstration layer needed to make those components inspectable.

The expected contribution is a documented and tested web-based or notebook-based workbench that improves development velocity, regression testing, artifact QA, and public demonstration of the symbolic-twin architecture.

Chapter 5

Integration Plan: Immediate Steps

The immediate goal is to establish a shared executable substrate for the project. The first prototype should not implement the full symbolic-twin architecture. It should produce a working BoxPush environment, a stable skill interface, a basic LM/DSPy skill-level controller, and logs that later components can use for symbolic modeling, monitoring, DBN learning, and belief update.

The current target platform is a BoxPush domain implemented on top of the continuous MiniGrid extension. The environment may use continuous movement and execution internally, but it must also expose the discrete information needed for symbolic reasoning. In particular, it must support full-state export for debugging and for Gilad’s first DBN-modeling steps.

5.1 M0: Shared BoxPush Platform, Skills, and Names

The purpose of M0 is to make sure that everyone is working with the same environment, the same skills, and the same names for shared objects, observations, execution labels, and trace fields. Pinhas will learn Fouzi’s current BoxPush environment and implement the required BoxPush functionality on top of the continuous MiniGrid version. Fouzi will learn to use this environment. Fouzi, Pinhas will jointly define the first BoxPush skill set and verify it with Ronen.

The M0 skill set should be small but sufficient for running simple BoxPush tasks. It should include only skills that can be implemented and tested immediately. Possible skills include navigation or positioning skills, pushing skills for light and heavy boxes, inspection skills, a delivery or goal-completion skill, and a declare-done skill. The exact list should be fixed jointly, with stable skill names, argument types, return values, and failure labels.

5.1.1 Shared Interface Artifacts

M0 must produce explicit interface artifacts that all components read or import. These artifacts define the common namespace for the first prototype. They are not yet the full symbolic model. They are the agreed interface layer that prevents Fouzi’s controller, Pinhas’s environment, Gilad’s DBN code, and Ron’s planning or recovery code from using incompatible names.

The following artifacts must be fixed during M0.

- **skill_registry**. The authoritative list of executable skills. For each skill it specifies the

skill name, argument names, argument types, callable signature, short natural-language description, possible return labels, and backend function.

- **object_registry.** The agreed names and types of BoxPush entities used in prompts, traces, state exports, and later symbolic models. This includes agents, boxes, goals, walls, regions, and any task-specific object categories.
- **state_schema.** The fields exported by the environment state. This should include agent state, box state, target or goal state, relevant spatial relations, and the discrete abstraction of the continuous environment.
- **observation_schema.** The observations returned during and after skill execution. This schema should distinguish raw environment outputs, interpreted progress observations for monitors, and terminal observations for belief update.
- **execution_labels.** A closed vocabulary of skill outcomes, such as **success**, **failure**, **timeout**, **invalid_skill**, **invalid_args**, **aborted**, and **declared_done**.
- **trace_schema.** The required fields in every logged step: task, state summary, selected skill, arguments, validation result, execution label, observations, natural-language execution summary, and full exported state.
- **environment_api.** The functions exposed by the environment, including **reset**, **observe**, **execute skill**, **export full state**, **export symbolic or discrete state**, and **terminate episode**.

These artifacts should be stored as explicit files or Python data structures, not only described in prose. A possible package layout is:

```
domains/boxpush/  
  skills.yaml  
  objects.yaml  
  state_schema.yaml  
  observations.yaml  
  execution_labels.yaml  
  trace_schema.yaml
```

All components should use these files rather than redefining names locally. Fouzi’s LM/DSPy controller should use **skill_registry** to construct the allowed skill list. Pinhas’s environment should use the same registry to expose executable skill hooks and return labels. Gilad’s DBN code should use **state_schema**, **observation_schema**, and **trace_schema**. Ron’s planning and recovery components should use the same skill names, state-variable names, and execution labels.

5.1.2 M0 Deliverables

The M0 deliverables are:

- a working continuous MiniGrid BoxPush environment;
- a fixed initial skill registry;
- executable implementations of the initial skills;
- stable skill signatures: name, arguments, types, and callable interface;
- stable skill return labels, such as **success**, **failure**, **timeout**, **invalid arguments**, or **aborted**;
- shared files or data structures for objects, state fields, observations, execution labels, and trace fields;
- full-state export after **reset**, during execution if useful, and at skill termination;